Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

VHDL-Übung 05: Sequenzielle Schaltungen und Strukturelle Beschreibung

M.Sc. Eike Trumann, M.Sc. Jasper Homann

Prof. Dr.-Ing. Guillermo Payá Vayá

2. Februar 2026

Aufgabe VHDL-500: *Parallel-zu-Seriell-Wandler*

VHDL-500

Während innerhalb von Schaltungen oft auf breite parallele Datenverbindungen zurückgegriffen werden kann, ist es für die Übertragung über längere Strecken häufig notwendig, Daten seriell, also auf derselben Leitung die unterschiedlichen Bits zeitlich nacheinander, zu übertragen. Hierfür werden sogenannte Parallel-zu-Seriell-Wandler benötigt.

In dieser Aufgabe soll ein 8-bit-Eingangsvektor seriell übertragen werden. Dabei soll aktuell die Bitrate auf ein Bit pro Taktzylus festgelegt werden, das Bit soll sich bei einer steigenden Taktflanke ändern.

Ein Problem mit der Übertragung von seriellen Daten ist die Synchronisation von Sender und Empfänger. Damit der Empfänger erkennen kann, dass ein Datenwert übertragen wird, führen wir eine zusätzliche Datenleitung mit dem Namen `transmission_o` ein. Diese soll auf `1` gesetzt werden, sobald Daten übertragen werden und auf `0`, sobald die Übertragung abgeschlossen ist. Dabei soll zwischen zwei übertragenenen 8-Bit-Datenworten eine Pause von mindestens einem Taktzyklus eingefügt werden, in der `transmission_o` auf 0 gesetzt ist. Die Übertragung soll mit dem niederwertigsten Bit `parallel_i(0)` beginnen.

Ein weiteres Problem ist, wenn sich ein Eingangsdatenwort während der Übertragung ändert. In der Implementierung soll es einen Speicher geben, der zu Beginn der Übertragung die Daten vom Eingang `parallel_i` übernimmt, sodass Änderungen während der Übertragung keinen Einfluss auf die aktuell übertragenen Daten haben sollen. Ein neues Datenwort soll genau dann übertragen werden, wenn sich der Wert von `parallel_i` vom vorherigen Wert unterscheidet und es Kapazität für die Übertragung gibt. Sollte sich

parallel_i während einer Übertragung mehrfach ändern, soll derjenige Wert übertragen werden, der in dem Moment, in dem die Übertragung des vorherigen Datenwortes abgeschlossen ist, am Eingang anliegt.

```
entity SERIALIZER is
port (
clk  : IN std_ulogic;
parallel_i : IN std_ulogic_vector(7 downto 0);
serial_o : OUT std_ulogic;
transmission_o : OUT std_ulogic);
end entity SERIALIZER;

architecture rtl of SERIALIZER is
  -- TODO
begin
 -- TODO
end rtl;
```

Aufgabe VHDL-501: *Seriell-zu-Parallel-Wandler*

Passend zu dem in Aufgabe VHDL-500 gezeigten Parallel-zu-Seriell-Wandler soll in dieser Aufgabe ein Seriell-zu-Parallel-Wandler erstellt werden, der die ursprünglichen parallelen Daten wiederherstellen kann.

VHDL-501

Dabei soll beachtet werden, dass während des Empfanges eines neuen Datenwortes das vorherige Datenwort weiterhin ausgegeben wird und das neue Datenwort erst am Ausgang erscheint, wenn die Übertragung abgeschlossen ist. Entsprechend muss es einen Speicher hierfür geben.

Da die Sendeeinheit ihre Daten bei einer steigenden Taktflanke verändert, ist es zweckmäßig, die Daten bei der darauf folgenden fallenden Taktflanke zu Lesen und zu speichern.

```
entity DESERIALIZER is
port (
clk  : IN std_ulogic;
serial_i : IN std_ulogic;
transmission_i : IN std_ulogic;
parallel_o : OUT std_ulogic_vector(7 downto 0)
);
end entity DESERIALIZER;

architecture rtl of DESERIALIZER is
  -- TODO
begin
 -- TODO
end rtl;
```

Aufgabe VHDL-502: *Schiebeoperation mit Multiplexern*

Wie wir bereits in der Übung gesehen haben, bieten VHDL-Dateien die Möglichkeit, mehrere Instanzen von Entities anzulegen.

In dieser Aufgaben wollen wir von dieser Funktialität gebrauch machen, um eine konfigurierbare Datenschiebeeinheit auf Basis von 4:1-Multiplexern zu erstellen.

Dabei soll die aus Aufgabe 403 bekannte Verschiebefunktion des `best_calculator` funktional identisch ausschließlich durch die Verwendung von Multiplexern implementiert werden.

Hierfür soll zunächst folgender 1-bit-Multiplexer implementiert werden:

```
entity MUX is
port(
  select_i    : in  std_ulogic_vector(1 downto 0);
  input_0_i   : in  std_ulogic;
  input_1_i   : in  std_ulogic;
  input_2_i   : in  std_ulogic;
  input_3_i   : in  std_ulogic;
  result_o    : out std_ulogic
);
end MUX;
```

Aus diesem Multiplexer soll dann eine Implementierung des `best_calculator` erstellt werden. Dabei soll die for-generate-Struktur von VHDL verwendet werden, um die Anzahl der ähnlichen Codezeilen zu reduzieren oder falls dieser benötigt wird, einen mehrbittigen Multiplexer abzubilden.

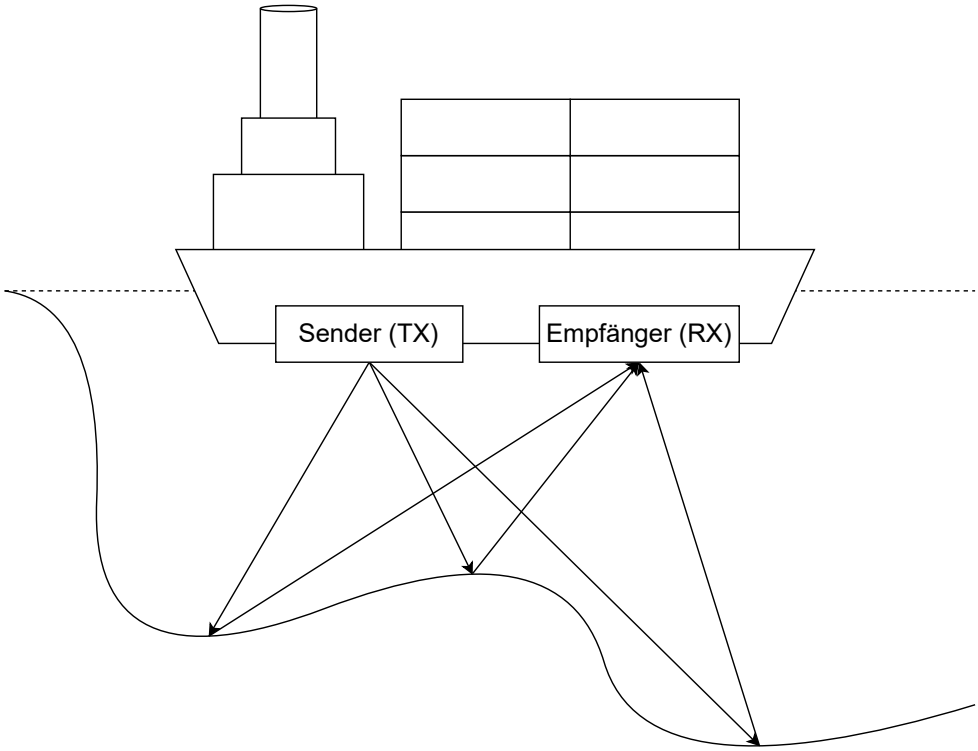
```
entity best_calculator is
port(
  data_bus     : in  std_ulogic_vector(7 downto 0);
  amount       : in  std_ulogic_vector(2 downto 0);
  logic_shift  : in  std_ulogic;
  left_shift   : in  std_ulogic;
  result       : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

Aufgabe VHDL-503: *Echelot*

In dieser Aufgabe soll die Auswertungsschaltung eines Echolots an Bord eines Schiffs implementiert werden. Unabhängig von der zu entwickelnden Schaltung werden automatisch Echo-Impulse ausgesendet. Abhängig von der Verzögerung und der Anzahl der aufgenommenen Reflexionen soll der Abstand bzw. die Beschaffenheit des Meeresbodens gemessen werden. Dabei gilt:

VHDL-503

- Die Verzögerung der ersten Reflexion steigt mit dem Abstand des Meeresbodens.
- Ein unebenerer Meeresboden führt zu mehr Reflexionen.



Es soll eine sequentielle Schaltung mit asynchronem Reset implementiert werden. Die zwei Eingangssignale `tx_i` und `rx_i` geben an, ob ein Echo-Impuls gesendet bzw. empfangen wurde und sind dabei jeweils einen Takt lang aktiv. Das erste `rx_i` tritt nie im gleichen Takt, wie das zugehörige `tx_i` auf.

Die drei Ausgabesignale der Schaltung sind:

- Der aktuell gemessene Abstand (`distance_o`)
- Die aktuell gemessenen Reflexionszahl (`texture_o`)
- Ein Fehlerausgang der bei mehr als 7 gemessenen Reflexionen aktiv ist (`error_o`)

Die Schaltung soll mithilfe von zwei Zählern implementiert werden, die beide auf 0 gesetzt werden, wenn ein neuer Impuls gesendet wird (`tx_i`). Der Zähler für den Abstand (Ausgabe auf `distance_o`) soll in jedem Takt bis inklusive dem der ersten Reflexion um 1 erhöht werden. Der Zähler für den Abstand soll bei jeder gemessenen Reflexion um 1 erhöht werden, dabei zählt es als 2 Reflexionen, wenn `rx_i` zwei Takte lang aktiv ist.

Es kann angenommen werden, dass die Zähler nie überlaufen.

```
entity echolot is
  port(
    areset_ni   : in  std_ulogic;
    clk         : in  std_ulogic;
    tx_i        : in  std_ulogic;
    rx_i        : in  std_ulogic;
```

```
    distance_o : out std_ulogic_vector(7 downto 0);
    texture_o  : out std_ulogic_vector(3 downto 0);
    error_o    : out std_ulogic
  );
end entity;
```

Implementieren Sie die Schaltung für die Auswertung des Echolots

Aufgabe VHDL-504: *BCD-Stoppuhr*

Beim BCD (Binary-Coded-Decimal) Format wird jede Ziffer einer Dezimalzahl mit je einer 4-Bit Binärzahl kodiert. Die Kodierungen mit einem Wert $\geq$ 10 werden dabei nicht verwendet. Das erleichtert den Entwurf eines System, in dem die Ausgabe ohnehin in Dezimalziffern erfolgt.

VHDL-504

In dieser Aufgabe soll eine 60-Sekunden Stoppuhr implementiert werden, die die Zeit mit zwei BCD kodierten Dezimalziffern ausgibt. Die Stoppuhr erhält als Eingang die Clock mit einem Taktfrequenz von 1 Hz, einen high-aktiven synchronen Reset und das Stoppsignal.

Sie gibt die beiden Dezimalziffern auf den Ausgängen `digit_1_o` und `digit_2_o` aus, wobei ersteres die Einerstelle und letzteres die Zehnerstelle angibt. Intern soll je ein 4-bit Register für die Stellen verwendet werden. Der Reset setzt die Zeit auf 0 Sekunden zurück.

Solange das Stoppsignal nicht aktiv ist soll die Zeit in jedem Takt um 1 erhöht werden. Sobald die Einerstelle den Wert 10 erreichen würde wird stattdessen die Zehnerstelle um 1 erhöht. Würde die Zehnerstelle den Wert 6 erreichen wird diese stattdessen auch auf 0 gesetzt.

```
entity stoppuhr is
  port(
    clk       : in  std_ulogic;
    reset_i   : in  std_ulogic;
    stop_i    : in  std_ulogic;
    digit_1_o : out std_ulogic_vector(3 downto 0);
    digit_2_o : out std_ulogic_vector(3 downto 0)
  );
  end entity;
```

Implementieren Sie die Stoppuhr

Aufgabe VHDL-505: *Bitzähler*

In dieser Aufgabe soll ein Decoder für ein 1-wire Übertragungsprotokoll implementiert werden. Auf der Leitung wird eine Sequenz von Symbolen (Zahlen zwischen 0 und 4) übertragen. Jedes Symbol wird dabei durch eine Folgen von einsen kodiert, deren Länge dem Wert des Symbols entspricht.

VHDL-505

Bitsequenz	Symbol
0	0
10	1
110	2
1110	3
11110	4

Kodierung der Symbole

Übertragene Bits	Dekodierte Symbole
110101110	2 1 3
10011000	1 0 2 0 0
1110111011	3 3

Beipielbitsequenzen und übertragene Symbole

Die Schaltung erhält als Eingang einen low-aktiven synchronen Reset (`reset_ni`) und das Signal `data_i` auf dem die Bitsequenz übertragen wird.

Während einsen übertragen werden soll die Schaltung einen Zähler inkrementieren. Dieser speichert also das Symbol das übertragen würde, wenn `data_i` im nächsten Takt 0 ist.

Sobald die Übertragung des Symbols durch eine 0 am Eingang `data_i` signalisiert wird, soll einen Takt später (!) der Ausgang `data_o` das übertragene Symbol ausgeben. Gleichzeitig soll der Ausgang `valid_o` genau einen Takt lang aktiv sein, um das Ende der Übertragung des Symbols zu signalisieren.

In allen anderen Takten muss das Signal `valid_o` inaktiv sein. Der Ausgang `data_o` hingegen darf beliebige Werte annehmen, solange `valid_o` inaktiv ist.

```
entity decoder is
  port(
    clk      : in  std_ulogic;
    reset_ni : in  std_ulogic;
    data_i   : in  std_ulogic;
    data_o   : out std_ulogic_vector(2 downto 0);
    valid_o  : out std_ulogic
  );
end entity;
```

Implementieren Sie die Dekoderschaltung für das Protokoll.